

abstract

Șapte exerciții — adaugi tipuri noi în ierarhiile tale de figuri și de orar · Java · Adrian

Unde lucrezi: repo-ul `java-abstractizare`. Ai copiat deja figurile; copiază la fel și intervalele de orar din proiectul de moștenire.

Nou, ai voie: `abstract` pe clasă și pe metodă, constructor `protected`.

Încă NU folosim: `interface`, `static`, `enum`, `record`, fișiere, `try/catch`, stream-uri. Vin separat, la rând.

Cum se lucrează: fiecare exercițiu îți cere să **construiești** ceva — o clasă nouă, o metodă mutată, o ierarhie reorganizată. Rândul **Gata când** spune cum verifici că ai terminat; se verifică prin **rulare**, nu prin citirea codului.

Partea 1 — adaugi un tip nou într-o ierarhie

1. Un triunghi în desen

nivel 1

Adaugă în ierarhia de figuri o clasă **Triunghi**, definită prin trei puncte. Trebuie să se poată afișa, muta și duplica exact ca celelalte figuri.

Înainte să-l scrii, fă `Figura` **abstractă** și declară abstracte cele trei metode pe care acum le are cu corp gol sau cu `return null`: afișarea, mutarea și duplicarea.

Apoi scrie `Triunghi` cu cele trei puncte și constructorul, și compilează. Compilatorul îți va spune ce mai are nevoie. Completează până compilează.

În `App`: pune triunghiul într-un `Desen` alături de celelalte figuri, afișează desenul, mută-l cu `(10, 5)`, afișează-l din nou.

Gata când: triunghiul apare în desen cu coordonatele lui, iar după mutare toate cele trei puncte s-au schimbat cu aceeași valoare.

De reținut: înainte, o figură scrisă pe jumătate compila liniștit și afișa un rând gol. Acum nu mai poți termina lucrul fără să scrii tot.

2. Un examen în orar

nivel 1

Același lucru, pe cealaltă ierarhie. Adaugă un tip nou de interval: **Examen** — ora de început, durata în minute, materia, sala și nota maximă.

Înainte să-l scrii, fă `Interval` **abstract** și declară abstractă metoda care dă tipul intervalului (acum întoarce șirul gol).

În `App`: construiește un orar cu un curs, o pauză, o activitate liberă și examenul, apoi afișează-l.

Gata când: orarul afișează patru rânduri, fiecare cu tipul lui scris corect, iar examenul își arată și materia, și sala, și nota maximă.

De reținut: dacă „uiți” metoda cu tipul, programul nu mai pornește deloc. Înainte, pornea și scria un rând fără tip.

Partea 2 — muți codul comun în clasa de sus

3. Formatul rândului, scris o singură dată

mai greu

Acum fiecare tip de interval își scrie singur rândul: metoda de afișare există în `Interval` și e rescrisă în toate subclasele, iar fiecare cheamă `super` și mai adaugă ceva.

Reorganizează așa: **rândul se compune într-un singur loc, în `Interval`**. Subclasele nu mai au voie să conțină metoda de afișare — ele dau doar bucățile care diferă (tipul și detaliile proprii), declarate abstract în `Interval`.

Ai de luat o decizie: **Pauza** n-are detalii proprii. Hotărăște ce întoarce ea și scrie într-un rând de ce.

Proba: adaugă un al cincilea tip, **Consultații** (ora, durata, profesorul, sala), și verifică.

Gata când: niciuna dintre subclase nu mai conține metoda de afișare; ieșirea pentru primele patru tipuri e identică cu cea de dinainte; iar rândul pentru `Consultații` apare corect **fără să fi modificat `Interval`**.

De reținut: clasa abstractă are voie și cu metode gata scrise, și cu metode doar declarate. Cele scrise le pot folosi pe cele declarate, chiar dacă nu știu încă ce vor face.

4. Ora și durata stau într-un singur loc

mai greu

Fă în așa fel încât ora de început și durata să existe **numai** în `Interval`: câmpuri `private`, constructor `protected`, iar subclasele să ajungă la ele doar prin getteri — niciuna nu le mai declară și niciuna nu le mai atinge direct.

Verifică apoi că `App` compilează și rulează neschimbat, deși constructorul lui `Interval` nu mai e public.

Gata când: `grep -n "oraInceput\|durataMin"` pe fiecare subclasă nu întoarce nimic, iar orarul afișează exact ce afișa înainte.

De reținut: o clasă abstractă nu e doar o listă de metode nescrise — ține și date, și constructor. Constructorul ei rulează de fiecare dată când creezi o subclasă, deși pe ea n-o poți crea.

5. Clasa din mijloc, pentru intervalele care au sală

mai greu

`OraCurs`, `Examen` și `Consultații` au toate o sală. `Pauza` și `ActivitateLibera` nu.

Bagă între ele o clasă intermediară care moștenește `Interval`, ține sala (câmp + getter, scrise o singură dată) și lasă nescrisă metoda cu detaliile — fiecare tip își spune detaliile altfel. Cele trei tipuri cu sală o moștenesc pe ea.

Gata când: cuvântul „sala” apare într-un singur fișier din ierarhie; orarul afișează la fel ca înainte; iar tu poți spune în scris care clase din ierarhie sunt abstracte și de ce fiecare.

De reținut: o clasă rămâne abstractă atâta timp cât mai are ceva nescris — indiferent câte metode a apucat să scrie.

6. Un desen în interiorul altui desen

provocare

Construiește un desen care conține, pe lângă figuri obișnuite, **un alt desen** (de exemplu o casă: un dreptunghi și un triunghi, băgate ca un tot într-un desen mai mare, lângă un cerc).

Verifică pe rând că:

- afișarea desenului mare arată și figurile desenului mic;
- mutarea desenului mare mută și figurile desenului mic, cu aceeași valoare;
- duplicarea desenului mare produce un desen complet independent — muți copia, originalul rămâne pe loc.

Gata când: afișezi originalul și copia înainte și după ce muți **doar copia**, și coordonatele originalului sunt neschimbate pe toate figurile, inclusiv pe cele din desenul mic.

De reținut: desenul mic e tratat ca orice altă figură. Codul care mută nu întreabă niciodată ce fel de figură are în față.

7. Tu decizi ce se declară abstract

provocare

Uită-te la ierarhia de figuri așa cum arată acum și hotărăște, pentru fiecare dintre cele trei metode din

Figura : rămâne scrisă acolo, cu un corp folositor pentru toată lumea, sau se declară abstractă și o scrie fiecare figură?

Scrie decizia și motivul, două rânduri pentru fiecare metodă.

Apoi pune-o la încercare: adaugă un **Romb** și scrie-l intenționat pe jumătate — doar câmpurile și constructorul. Compilează.

Gata când: compilatorul te oprește exact pe metodele pe care tu le-ai declarat abstracte, nici mai multe, nici mai puține — și asta se potrivește cu ce ai scris în motivare.

De reținut: ce declari abstract devine o obligație pentru oricine vine după tine în ierarhie. E o decizie de proiectare, nu o regulă de sintaxă.